R: limpieza, manipulación y exploración de datos

Cheatsheet de `dplyr`

Jonatán Perren

24 de marzo de 2026

gxono

El paquete `dplyr`

Una gramática para la manipulación de datos

`dplyr` es el paquete central del tidyverse para transformar datos tabulares. Trabaja sobre data frames y tibbles.

Los 6 verbos fundamentales

- `select()` — Elegir columnas.
- `filter()` — Filtrar filas por condición.
- `mutate()` — Crear o modificar columnas.
- `arrange()` — Ordenar filas.
- `summarise()` — Resumir datos.
- `group_by()` — Agrupar para operar por grupo.

Todas comparten la misma estructura: el primer argumento es el data frame y el resultado es un nuevo data frame. Esto permite encadenarlas con el pipe $\triangleright$.

glimpse: Vistazo rápido al dataset

`glimpse()` es una alternativa a `str()` adaptada al tidyverse. Muestra la estructura de la tabla transponiendo filas por columnas, ideal para un primer vistazo.

```r
glimpse(starwars)
```

```
Rows: 87
Columns: 14
$ name       <chr> "Luke Skywalker", "C-3PO", "R2-D2", "Darth Vader" ...
$ height     <int> 172, 167, 96, 202, 150, 178, 165, 97, 183, 182 ...
$ mass       <dbl> 77.0, 75.0, 32.0, 136.0, 49.0, 120.0, 75 ...
```

select: Elegir columnas

`select()` elige, reordena o renombra columnas.

```
library(dplyr)

starwars ▷ select(name, height, mass)
```

Excluir columnas

```
starwars ▷ select(-hair_color, -skin_color)
```

Seleccionar rangos

```
starwars ▷ select(name:mass)        # De name a mass
starwars ▷ select(1:3)              # Primeras 3 columnas
```

Renombrar al seleccionar

```
starwars ▷ select(nombre = name, altura = height)
```

Para renombrar sin descartar columnas, usá `rename()`:

```
starwars ▷ rename(nombre = name)
```

Selectores auxiliares (tidyselect)

Los selectores permiten elegir columnas por patrón, sin nombrarlas una a una.

Selector	Descripción
`starts_with("x")`	Columnas que empiezan con `"x"`
`ends_with("x")`	Columnas que terminan con `"x"`
`contains("x")`	Columnas que contienen `"x"`
`matches("regex")`	Columnas que matchean una regex
`num_range("x", 1:3)`	Columnas `x1, x2, x3`
`everything()`	Todas las columnas restantes
`last_col()`	Última columna
`where(fn)`	Columnas donde `fn` devuelve `TRUE`
`all_of(vector)`	Todas las columnas en el vector (error si falta una)
`any_of(vector)`	Las que existan del vector (no da error)

Selectores: Ejemplos

```r
starwars ▷ select(starts_with("h"))
# height, hair_color, homeworld

starwars ▷ select(ends_with("color"))
# hair_color, skin_color, eye_color
```

Seleccionar por tipo o condición

```r
starwars ▷ select(where(is.numeric))
starwars ▷ select(where(\(x) is.numeric(x) && mean(x, na.rm = T) > 100))
```

Negar selectores (excluir)

```r
starwars ▷ select(!starts_with("h"))
starwars ▷ select(!where(is.character))
```

Combinar selectores

```r
# Nombre + todas las numéricas
starwars ▷ select(name, where(is.numeric))
```

Seleccionar desde un vector de nombres

```r
cols <- c("name", "height", "mass")
starwars |> select(all_of(cols))
```

Los selectores también funcionan en `across()`, `rename_with()`, `pivot_longer()` y otras funciones del tidyverse.

filter: Filtrar filas

`filter()` conserva las filas que cumplen una condición lógica.

```
starwars ⊳ filter(height > 180)
```

Múltiples condiciones

```
# AND: separar con coma o &
starwars ⊳ filter(height > 180, mass < 100)
starwars ⊳ filter(height > 180 & mass < 100) # Equivalente

# OR: usar |
starwars ⊳ filter(species == "Human" | species == "Droid")
```

Filtrar con `%in%`

```
starwars ⊳ filter(species %in% c("Human", "Droid"))
```

Negar condiciones

```
starwars ⊳ filter(!is.na(height))
starwars ⊳ filter(!(species %in% c("Human", "Droid")))
```

filter: Con cadenas de texto

```r
# Filtrar filas que contengan un patrón
starwars ⊳ filter(str_detect(name, "Skywalker"))

# Filtrar filas que NO contengan el patrón (negar)
starwars ⊳ filter(!str_detect(name, "Skywalker"))
```

between(): Rango numérico

```r
starwars ⊳ filter(between(height, 160, 180))
```

near(): Comparación con tolerancia

```r
starwars ⊳ filter(near(mass, 80, tol = 5))
# Filas donde mass está entre 75 y 85
```

mutate: Crear y modificar columnas

`mutate()` agrega nuevas columnas o modifica las existentes.

```
starwars ▷ mutate(
  imc = mass / (height / 100)^2,
  altura_m = height / 100
)
```

Referir columnas recién creadas

```
starwars ▷ mutate(
  altura_m = height / 100,
  imc = mass / altura_m^2 # Usa altura_m, recién definida
)
```

.before y .after: Posicionar la columna nueva

```
starwars ▷ mutate(
  imc = mass / (height / 100)^2,
  .after = mass
)
```

Funciones de ventana: lag y lead

Permiten mirar valores anteriores (`lag()`) o posteriores (`lead()`). Fundamentales en series de tiempo y datos ordenados secuencialmente.

```r
x <- c(1, 2, 3, 4, 5)
```

```
`:>:> lag(x)`
[1] NA  1  2  3  4
`:>:> lead(x)`
[1]  2  3  4  5 NA
```

```r
# Asumimos que la tabla de ventas está ordenada cronológicamente
ventas %> mutate(
  monto_anterior = lag(monto),
  crecimiento = monto - lag(monto)
)
```

Si los aplicás sobre datos agrupados (`group_by()`), el rezago respetará los límites de cada grupo (ej: mes anterior del mismo cliente o región).

Sobreescribir una columna

```r
starwars ▷ mutate(height = height / 100)
```

.keep: Controlar qué columnas conservar

```r
# .keep = "used": solo conserva las columnas usadas o creadas
starwars ▷ mutate(
  imc = mass / (height / 100)^2,
  .keep = "used"
)
```

mutate con condicionales: if_else y case_when

if_else: Dos ramas

```
starwars ▷ mutate(
  categoria = if_else(height > 180, "Alto", "Bajo")
)
```

case_when: Múltiples condiciones

Evalúa de arriba hacia abajo; la primera condición verdadera gana.

```
starwars ▷ mutate(
  rango = case_when(
    height > 200 ~ "Muy alto",
    height > 180 ~ "Alto",
    height > 160 ~ "Medio",
    .default = "Bajo"
  )
)
```

case_match: Recodificar valores exactos

```
starwars ▷ mutate(
  especie = case_match(species,
    "Human"     ~ "Humano",
    "Droid"     ~ "Droide",
    "Wookiee"   ~ "Wookiee",
    .default = "Otro"
  )
)
```

arrange: Ordenar filas

`arrange()` ordena las filas según una o más columnas.

```
# Ascendente (por defecto)
starwars ▷ arrange(height)

# Descendente
starwars ▷ arrange(desc(height))

# Múltiples criterios: primero por species, luego por height desc.
starwars ▷ arrange(species, desc(height))
```

Los NA siempre se colocan al final, sin importar el orden.

summarise: Resumir datos

`summarise()` colapsa el data frame a una fila resumen.

También se puede colapsar una fila por grupo si se usa con `group_by()`.

```r
starwars ▷ summarise(
  media_altura = mean(height, na.rm = TRUE),
  max_masa = max(mass, na.rm = TRUE),
  n = n()
)
```

```
# A tibble: 1 x 3
  media_altura max_masa     n
         <dbl>    <dbl> <int>
1         175.     1358    87
```

Funciones de resumen útiles

Función	Descripción
`n()`	Cantidad de filas
`n_distinct(x)`	Valores únicos
`sum(x) / mean(x)`	Suma / Promedio
`min(x) / max(x)`	Mínimo / Máximo
`sd(x) / var(x)`	Desvío estándar / Varianza
`median(x)`	Mediana
`first(x) / last(x)`	Primer / Último valor

group_by + summarise

`group_by()` define grupos; los verbos subsiguientes operan por grupo.

```
starwars ▷
  group_by(species) ▷
  summarise(
    n = n(),
    altura_media = mean(height, na.rm = TRUE)
  ) ▷
  arrange(desc(n))
```

```
# A tibble: 38 x 3
   species      n altura_media
   <chr>    <int>        <dbl>
 1 Human       35         177.
 2 Droid        6         131.
 3 NA           4         175
 4 Gungan       3         209.
...
```

Por defecto, `summarise()` elimina el **último nivel** de agrupamiento y lanza un mensaje. Para controlarlo explícitamente, usá el argumento `.groups`: `"drop"` (elimina todos), `"keep"` (mantiene todos) o `"drop_last"`.

Agrupación temporal con .by

Desde `dplyr 1.1.0`, podés agrupar directamente dentro de `summarise()`, `mutate()` o `slice()` usando el argumento `.by`.

Esto evita tener que hacer el `group_by()` previo y el `ungroup()` posterior, previniendo errores silenciosos.

```
# Mucho, mucho más seguro y directo
starwars ▷ summarise(
  n = n(),
  altura_media = mean(height, na.rm = TRUE),
  .by = species
)

# Con mutate
ventas ▷ mutate(pct_region = monto / sum(monto), .by = region)
```

group_by + mutate

Al usar `mutate()` sobre datos agrupados, las funciones de resumen operan **dentro de cada grupo** sin colapsar filas.

```
starwars ⊳
  group_by(species) ⊳
  mutate(
    altura_media = mean(height, na.rm = TRUE),
    diff = height - altura_media
  ) ⊳
  ungroup()
```

Esto es muy útil para calcular proporciones dentro de cada grupo:

```
ventas ⊳
  group_by(region) ⊳
  mutate(
    pct = monto / sum(monto) * 100
  ) ⊳
  ungroup()
```

Siempre llamá a `ungroup()` cuando termines de operar con grupos, para evitar comportamientos inesperados en operaciones posteriores.

Contar: count y tally

`count()` es un atajo para `group_by() ⊳ summarise(n = n())`.

```
starwars ⊳ count(species, sort = TRUE)
```

```
# A tibble: 38 x 2
   species      n
   <chr>    <int>
 1 Human       35
 2 Droid        6
 3 NA           4
 4 Gungan       3
...
```

```
# Con wt: en vez de contar filas, SUMA una columna
starwars ⊳ count(species, wt = mass, sort = TRUE,
                 name = "masa_total")
```

```
# A tibble: 38 x 2
   species  masa_total
   <chr>         <dbl>
 1 Hutt           1358
 2 Human          1886
 3 Kaminoan        178
...
```

Sin `wt`, `count()` suma `1` por cada fila que pertenece al grupo (cuenta cantidad de observaciones).

Con `wt = mass`, en lugar de sumar `1`, **suma el valor** de la columna `mass` para cada observación de ese grupo. Es el equivalente a hacer:

```
starwars ⊳
  group_by(species) ⊳
  summarise(masa_total = sum(mass, na.rm = TRUE)) ⊳
  arrange(desc(masa_total))
```

`add_count()` agrega la columna `n` sin colapsar filas. Hacer `starwars ⊳ add_count(species)` es equivalente a:

```
starwars ⊳
  group_by(species) ⊳
  mutate(n = n()) ⊳
  ungroup()
```

across: Aplicar funciones a múltiples columnas

`across()` permite aplicar una o más funciones a un conjunto de columnas seleccionadas con tidyselect.

```r
# Redondear todas las columnas numéricas
starwars ▷ mutate(
    across(where(is.numeric), round, digits = 1)
  )
```

```r
# Modificar todas las columnas (everything)
starwars ▷ mutate(
    across(everything(), as.character)
  )
```

```r
# Resumir mín y máx de las columnas numéricas
starwars ▷ summarise(
    across(where(is.numeric),
    list(min = ~min(.x, na.rm = TRUE),
          max = ~max(.x, na.rm = TRUE)))
  )
```

Funciones anónimas con ~ (fórmula) o

```r
# Con fórmula ~: .x es el placeholder
starwars ▷ mutate(
    across(where(is.character), ~str_to_title(.x))
  )

# Con función anónima \(x):
starwars ▷ mutate(
    across(where(is.character), \(x) str_to_title(x))
  )
```

across: Renombrar resultados

Cuando se aplican múltiples funciones, `.names` controla el nombre de las columnas resultantes.

```r
starwars ▷ summarise(
  across(c(height, mass),
    list(media = ~mean(.x, na.rm = TRUE),
         sd = ~sd(.x, na.rm = TRUE)),
    .names = "{.col}_{.fn}")
)
```

```
# A tibble: 1 x 4
  height_media height_sd mass_media mass_sd
         <dbl>     <dbl>      <dbl>   <dbl>
1         175.      34.8       97.3    169.
```

across en filter

```r
# Filas donde TODAS las numéricas son no-NA
starwars ▷ filter(
  if_all(where(is.numeric), ~!is.na(.x))
)

# Filas donde ALGUNA numérica es > 100
starwars ▷ filter(
  if_any(where(is.numeric), ~.x > 100)
)
```

rowwise: Operaciones horizontales

Por defecto, `mutate()` y `summarise()` operan por columna (vectorialmente). Si necesitás operar a lo ancho (evaluar fila por fila), usá `rowwise()`.

```
# Imaginá un dataset con notas de 3 parciales: p1, p2, p3
notas ▷
  rowwise() ▷
  mutate(
    promed = mean(c_across(c(p1, p2, p3)), na.rm = TRUE)
  ) ▷
  ungroup() # Importante: remueve el modo "por fila"
```

`c_across()` sirve para seleccionar las columnas horizontales usando la misma sintaxis que `select()` (como `starts_with()` o `where(is.numeric)`).

slice: Seleccionar filas por posición

Familia de funciones para elegir filas específicas.

```
starwars ⊳ slice(1:5)  # Primeras 5 filas
starwars ⊳ slice_head(n = 3)
starwars ⊳ slice_tail(n = 3)
```

slice_min y slice_max

```
# Los 5 personajes más altos (trae más si hay empates)
starwars ⊳ slice_max(height, n = 5)

# Omitir empates
starwars ⊳ slice_max(height, n = 5, with_ties = FALSE)
```

Corte por grupos

```
# Los 3 más livianos por especie
starwars ⊳
  group_by(species) ⊳
  slice_min(mass, n = 3) ⊳
  ungroup()
```

slice_sample: Muestra aleatoria

```
starwars ⊳ slice_sample(n = 10)
starwars ⊳ slice_sample(prop = 0.1)
```

distinct: Valores únicos

```r
starwars ▷ distinct(species)
```

```
# A tibble: 38 x 1
   species
   <chr>
 1 Human
 2 Droid
 3 Wookiee
 ...
```

Conservar las demás columnas:

```r
starwars ▷ distinct(species, .keep_all = TRUE)
```

pull: Extraer una columna como vector

Dejar solo la columna extraída:

```
starwars ⊳
  filter(species == "Human") ⊳
  pull(name)
```

```
[1] "Luke Skywalker"        "Darth Vader"  ...
```

Extraer asignando nombres (vector con nombre):

```
starwars ⊳
  filter(species == "Human") ⊳
  pull(height, name = name)
```

```
Luke Skywalker    Darth Vader  ...
172            202  ...
```

relocate y rename_with

relocate: Mover columnas

```
starwars ▷ relocate(species, .before = name)
starwars ▷ relocate(homeworld, .after = name)
starwars ▷ relocate(where(is.numeric), .after = name)
```

rename_with: Renombrar con una función

```
# Todo a minúsculas
starwars ▷ rename_with(tolower)

# Reemplazar puntos por guiones bajos
iris ▷ rename_with(~str_replace_all(.x, "\\.", "_"))
```

```
# Columnas: sepal_length, sepal_width, ...
```

```
# Solo renombrar las numéricas
starwars ▷ rename_with(str_to_upper,
                       .cols = where(is.numeric))
```

Joins: Unir tablas

Las funciones `*_join()` combinan dos tablas por columnas en común.

Función	Descripción
`left_join(x, y)`	Todas las filas de x, matcheando con y
`right_join(x, y)`	Todas las filas de y, matcheando con x
`inner_join(x, y)`	Solo filas con match en ambas tablas
`full_join(x, y)`	Todas las filas de ambas tablas
`semi_join(x, y)`	Filas de x que tienen match en y (sin agregar columnas)
`anti_join(x, y)`	Filas de x que NO tienen match en y

Joins: Ejemplos

```r
alumnos <- tibble(
  id = 1:4,
  nombre = c("Ana", "Luis", "María", "Pedro")
)

notas <- tibble(
  id = c(1, 2, 2, 4, 5),
  materia = c("Macro", "Micro", "Macro", "Macro",
              "Estadística"),
  nota = c(8, 7, 9, 6, 10)
)
```

```r
alumnos |> left_join(notas, by = "id")
```

```
# A tibble: 5 x 4
     id nombre materia nota
  <int> <chr>  <chr>  <dbl>
1     1 Ana    Macro      8
2     2 Luis   Micro      7
3     2 Luis   Macro      9
4     3 María  NA        NA
5     4 Pedro  Macro      6
```

María (id=3) aparece con `NA` porque no tiene notas. El id=5 de notas no aparece porque `left_join` conserva solo las filas de la tabla izquierda.

Joins: Claves diferentes y semi/anti

Cuando las columnas tienen distinto nombre

```
alumnos ⊳
  left_join(notas, by = join_by(id == alumno_id))
```

semi_join: Filtrar sin agregar columnas

```
# Alumnos que tienen al menos una nota
alumnos ⊳ semi_join(notas, by = "id")
```

```
# A tibble: 3 x 2
     id nombre
1     1 Ana
2     2 Luis
3     4 Pedro
```

anti_join: Encontrar lo que NO matchea

```
# Alumnos sin notas
alumnos ⊳ anti_join(notas, by = "id")
```

```
# A tibble: 1 x 2
     id nombre
1     3 María
```

Combinar tablas: bind y operaciones de conjunto

bind_rows: Apilar tablas verticalmente

```r
cuatri_1 <- tibble(materia = "Macro", nota = c(7, 8))
cuatri_2 <- tibble(materia = "Micro", nota = c(9, 6))

bind_rows(cuatri_1, cuatri_2)
```

```
# A tibble: 4 x 2
  materia  nota
  <chr>   <dbl>
1 Macro       7
2 Macro       8
3 Micro       9
4 Micro       6
```

```r
# Con identificador de origen
bind_rows(cuatri_1, cuatri_2, .id = "cuatrimestre")
```

Si una tabla tiene columnas que la otra no, se rellenan con `NA`.

bind_cols: Pegar columnas

```r
bind_cols(nombres, edades) # Deben tener mismo n de filas
```

El pipe: Encadenar operaciones

El pipe ⊳ pasa el resultado de la izquierda como primer argumento de la función de la derecha.

```r
# Sin pipe (difícil de leer)
arrange(
  summarise(
    group_by(
      filter(starwars, !is.na(height)),
      species),
    media = mean(height)),
  desc(media))

# Con pipe (flujo natural)
starwars ⊳
  filter(!is.na(height)) ⊳
  group_by(species) ⊳
  summarise(media = mean(height)) ⊳
  arrange(desc(media))
```

⊳ es el pipe nativo de R (v4.1+). El tidyverse también ofrece `%>%` del paquete `magrittr`. Ambos funcionan igual para los usos más comunes.

Manejo de NA en dplyr

Filtrar NA

```r
# Eliminar filas con NA en una columna
starwars ▷ filter(!is.na(height))

# Eliminar filas con NA en cualquier columna seleccionada
starwars ▷ filter(
  if_all(c(height, mass), ~!is.na(.x))
)
```

drop_na (de tidyr)

```r
starwars ▷ tidyr::drop_na(height, mass)
```

Reemplazar NA con valores

```r
# Para una sola columna: coalesce (primer valor no-NA)
starwars ▷ mutate(
  mass = coalesce(mass, 0)
)

# Para múltiples columnas o usando tidyr
starwars ▷ mutate(
  across(where(is.numeric), ~coalesce(.x, 0))
)
# starwars |> tidyr::replace_na(list(`:bn:mass` = 0, `:bn:height` = 0))
```

Resumen de verbos

Inspección

`glimpse()` (como un str() moderno).

Filas

`filter()`, `arrange()`, `slice()`, `slice_head()`, `slice_tail()`, `slice_min()`, `slice_max()`, `slice_sample()`, `distinct()`.

Columnas

`select()`, `rename()`, `rename_with()`, `relocate()`, `mutate()`, `pull()`.

Agrupación y resumen

`group_by()`, `ungroup()`, `summarise()`, `count()`, `add_count()`, `tally()`. (Acordate del argumento `.by`!)

Condicionales y Ventanas

`if_else()`, `case_when()`, `case_match()`, `coalesce()`, `lag()`, `lead()`.

Iteración y Operaciones Horizontales

`across()`, `if_all()`, `if_any()`, `rowwise()`, `c_across()`.

Uniones

`left_join()`, `right_join()`, `inner_join()`, `full_join()`, `semi_join()`, `anti_join()`, `bind_rows()`, `bind_cols()`.